

CampusCare

Approach Note & Engineering Decisions

Assignment 4 — Student Support & Ticket Management

Submission to: `tech_interview@edumerge.com`

Candidate Submission Note

1. Problem Understanding

Educational institutions handle thousands of student inquiries each semester across multiple administrative bodies (Accounts, Academic Affairs, Registry, IT Support, Facilities). When these requests are handled via informal emails, paper slips, or in-person office visits:

- Requests get lost or overlooked with no central record.
- Students face immense anxiety because they have no visibility into who is working on their issue or when it will be resolved.
- Administrative staff have unbalanced workloads and unclear ownership of tickets.
- Urgencies (such as exam hall ticket lockouts or fee payment deadlines) get buried under routine queries.
- Management has zero real-time metrics on staff responsiveness, ticket ageing, or SLA breach trends.

Our Goal: Build **CampusCare**, a centralized, traceable, full-stack platform that replaces communication clutter with structured ticket lifecycles, clear ownership, role privacy, and automated SLA deadline monitoring.

2. Product Thinking & Key Assumptions

- Single-Institution, Multi-Department Scope:** The platform serves one college or university with multiple specialized departments (Finance, Academics, Registry, IT, General Admin).
- Distinct Role Boundaries:**
 - Student:** Needs a clean self-service portal to submit inquiries, track progress, respond to staff requests, and reopen unresolved issues. They must never see other students' requests or private staff discussions.
 - Support Staff:** Needs an organized queue, clear SLA deadlines, the ability to post public replies to students and private notes to colleagues, and a clear resolution mechanism.
 - Admin / Manager:** Needs high-level institutional oversight, the ability to balance staff workloads, manage ticket categories and SLA policies, and export data for reporting.
- SLA Deadlines Based on Elapsed Time:** For demo clarity and evaluation, SLA deadlines count down based on real clock time across four tiers:
 - `Urgent` : 30 min first response / 4 hr resolution
 - `High` : 2 hr first response / 24 hr resolution
 - `Medium` : 8 hr first response / 48 hr resolution
 - `Low` : 24 hr first response / 72 hr resolution
- Action Ownership:** Every ticket displays whose court the ball is in (`next_action_owner` is Student, Staff, or Manager).

3. Technical Approach & Engineering Decisions

Decision	Choice Made	Rationale & Practicality
Backend Framework	Node.js with Express	Lightweight, fast setup, ubiquitous, and straightforward REST route architecture.
Database Engine	SQLite via <code>better-sqlite3</code>	Zero-configuration, file-based relational database that persists on disk (<code>server/campuscare.db</code>). Runs synchronously in Node, preventing asynchronous query drift and race conditions during transactional writes. Enabled Write-Ahead

Logging (WAL) and foreign key constraints.		
Authentication & RBAC	JWT (JSON Web Tokens) & bcryptjs	Stateless authentication with passwords salted and hashed. Server-side middleware validates the user's role on every sensitive operation rather than relying on frontend UI visibility alone.
Frontend Framework	React 19 + Vite	Fast HMR dev server, component-based modular UI, and quick production bundling (compiled in < 1 second).
Styling Strategy	Vanilla CSS Design System (src/index.css)	Avoided heavy CSS framework lock-ins. Crafted a clean SaaS interface using CSS custom properties, Google Fonts (Plus Jakarta Sans & Inter), responsive layouts, and distinct status color tokens.
Data Visualizations	Custom Pure SVG Charts (Charts.jsx)	Lightweight, dependency-free Donut charts, Bar charts, and Ageing histograms that render with zero external bundle bloat.

4. Key Edge Cases & Failure Scenarios Handled

1. Student Data Isolation (Unauthorized Direct API Access)

SCENARIO: A student modifies a URL or makes an API request to view ticket #2 belonging to another student.

DEFENSE: Server validates `requester_id === req.user.id` and rejects with `HTTP 403 Forbidden`.

2. Internal Notes Information Leak

SCENARIO: Staff member posts confidential discussion regarding fee reconciliation.

DEFENSE: The `/api/tickets/:id` query automatically filters out `visibility = 'internal'` records if the requesting user has the `student` role.

3. Empty Resolution Summary Prevention

SCENARIO: Staff attempts to mark a ticket as `Resolved` without explaining what was done.

DEFENSE: Both the frontend modal and the backend route reject the status transition with `HTTP 400` if the resolution summary is missing or under 5 characters.

4. Automatic Ticket Re-Activation on Student Reply

SCENARIO: A ticket is in `Waiting for Student`. The student submits their requested document or clarification.

DEFENSE: The comment creation transaction automatically updates ticket status back to `In Progress` and switches `next_action_owner` to `Staff`.

5. SLA Recalculation on Priority Change

SCENARIO: A manager escalates a ticket from `Low` to `Urgent`.

DEFENSE: The system recalculates `resolution_due_at` from the original creation timestamp using the new policy and logs the adjustment in the audit history.

6. Browser CSV Download Authentication

SCENARIO: Browser `<a href="..." download>` clicks cannot send custom authorization headers.

DEFENSE: The authentication middleware inspects both `Authorization` headers and `?token=` query parameters.

5. Architectural Trade-offs & Limitations

1. SQLite vs. Client-Server Database (PostgreSQL)

TRADE-OFF: Chosen for standalone demonstrability without requiring Docker or cloud DB setup during review.

LIMITATION: For production scaling across tens of thousands of concurrent writes, migrating to PostgreSQL with a connection pooler would be the next step.

2. Elapsed Clock vs. Business Hours SLA

TRADE-OFF: Used continuous elapsed clock time for clear prototype demonstration. In an enterprise deployment, an institution holiday and business hours calendar (e.g., 9 AM to 5 PM weekdays) would be factored in.

3. In-App Notifications vs. External Email/SMS

TRADE-OFF: Implemented an in-app notification center with unread badges to keep the prototype self-contained without needing third-party Twilio or SendGrid API keys.

6. Verification Summary

- **Automated Validation:** `npm test` runs 20 automated tests covering scenarios T-001 through T-020 (100% pass rate).
- **Manual Checklist:** Documented in `SRS_TEST_CASES.md`.
- **AI Usage Documentation:** Fully documented in `AI_USAGE_REPORT.md`.